

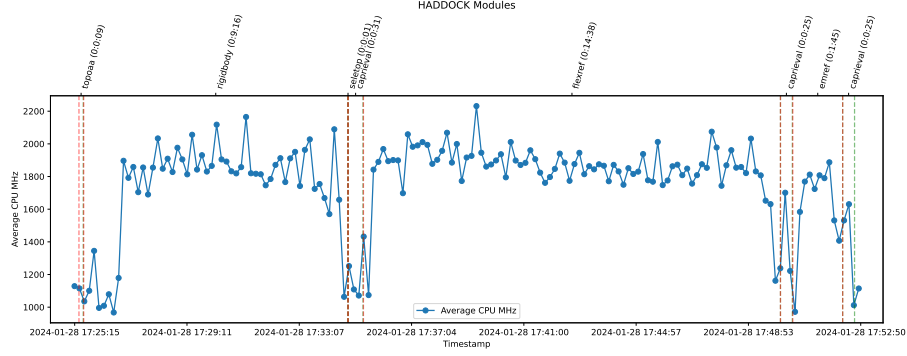


Figure 1: Overall CPU Frequency (Mhz) Trace for DPP executed with 16 active vCPUs.

1 Tactics

This document describes the 8 tactics defined in the first phase of the study. Each subsection is dedicated to a tactic and deepens the definition, application context, and assessment by the HADDOCK teams.

1.1 T1 Choose a Suitable DVFS Policy

Short description – Select the DVFS policy at the OS level based on the expected computational demands of the modules to be executed.

Context – Our analysis of the performance impact of individual modules revealed which modules tend to consume more resources. Figure 1 shows a decrease in the average vCPU frequency for modules dedicated to lightweight tasks, such as `topoaa` and `caprieval`. The `topoaa` module creates all-atom topologies, while `caprieval` calculates CAPRI metrics. This phenomenon occurs because of the CPU governor value set to `ondemand` in Ubuntu during the experiment. We controlled the CPU governor during our experiment and set it to `ondemand` since it is the default governor in many kernels [18]. This governor rapidly adjusts the CPU frequency to a value proportional to the current load or to the maximum CPU frequency when the load exceeds a certain threshold [38]. In both cases, the adjustment is immediate. This aggressive policy efficiently handles workload increases for better performance, but consumes more power by quickly ramping up to higher frequencies, leading to higher overall energy consumption.

Description – The development team shall consider a frequency scaling policy that is (i) tailored to the characteristics of the modules of their system and (ii) more conservative compared to the default `ondemand` policy. Possible implementations of this policy include: (i) scaling the frequency gradually, for example using the `conservative` governor [38], (ii) avoiding frequency scaling for short-lived tasks, and (iii) in the case of parallel modules execution, allocat-

ing a subset of vCPUs to modules requiring more resources and increasing the frequency of exclusively those vCPUs.

Assessment – The HADDOCK team is quite diffident concerning the *applicability* of this tactic because, in their specific context, the workload is distributed in such a way to try to always fill all available computing resources. Therefore, for the HADDOCK team, the best solution is to implement a Race-to-Idle policy and fix the frequency at its maximum value to finish the computation as soon as possible. Nevertheless, the HADDOCK team acknowledges the *low effort* to integrate this tactic in their setting as it requires only to set a parameter of the kernel (assuming that a proper runtime monitoring infrastructure is already up, and provides an indication of the resources consumed by each module). The HADDOCK team has doubts regarding the implementation of the proposed tactic, especially in cases of multi-tenancy. In addition, the software developer is concerned about the overhead introduced for polling and setting the frequency of the vCPUs for dynamically changing loads. The tactic is deemed as *original* for the domain expert and the software developer, while it is *conventional* for the product owner. Finally, during the focus group the product owner stressed several times the importance of evaluating the trade-offs between performance and energy efficiency according to the type of load, namely to evaluate if a performance boost leads to an improvement in terms of energy efficiency or not. By quoting the product owner:

“Using more cores can increase the throughput, but will it decrease the energy? We can make a simple experiment: you can execute at the maximum speed, let us say 100%, and then repeat the execution running at 90% or 80% of the speed. It will take 20% more time, but which of the two scenarios allows you to save energy?” - Product Owner

The literature provides evidence of the benefits of changing CPU frequency policy using the OS [14, 18, 32]. Huang *et al.* [14] confirm our suggestion to adapt CPU frequency at runtime according to workloads that are application-specific. The authors develop GreenDVFS, which involves machine learning to predict the workload distribution and change CPU frequency.

1.2 T2 Scale vCPU Frequency by Workload

Short description – Proactively tune the frequency of vCPUs based on the current computational demands of the modules being executed.

Context – Bao et al. [2] show that commonly used dynamic frequency scaling approaches may not effectively reduce the energy consumption of software. The energy benefits caused by throttling the CPUs to the maximum or minimal frequency may vary according to the voltage and frequency range of the CPUs. The authors demonstrate that running the same benchmark while using all the cores can achieve minimal energy on the SandyBridge and Haswell CPU architectures by setting the minimal frequency (1.6 GHz). In contrast,

when using two Ivy Bridge processors, the minimum energy consumption is obtained near the maximum frequency supported by the processor. In addition, a frequency scaling policy should also consider the characteristics of the running application. For example, for bandwidth-bound cases, namely, workloads that involve many memory accesses, the CPU is often stalled waiting for data from memory. Therefore, an optimal frequency scaling mechanism should be specific to the workload type rather than based on the expected CPU load, as current approaches do.

Description – The frequency of the vCPUs shall be tuned at runtime based on the characteristics of the modules of the running system. Coming back to HADDOCK, the modules of HADDOCK have different functions, ranging from calculating metrics, selecting a model subset, and matrix multiplications. The workload generated by the modules of HADDOCK can drastically vary based on the functionalities of a specific module. As shown in Figure 1 different modules generate different CPU frequency. We suggest implementing a different frequency scaling policy for each module based on its characteristics. Such a policy can be defined based on historical data (*i.e.*, proactively) retrieved during previous executions of the system. In this way, as in Bao et al., such a policy would be tailored to the type of workload and, thus, reduce energy waste and increase the probability of achieving optimal energy efficiency. A custom frequency scaling policy could consider not only individual modules but also their sequences and combinations.

This tactic seems similar to T1. The tactics are different in the actor adjusting the frequency of the vCPUs. For T1, the user (or researcher) sets the frequency based on his experience as he might be interested in executing a particular combination of modules of HADDOCK. In T2, the software adopts a self-adoptive behavior towards the computational loads and scales the frequency accordingly. For example, T2 addresses scenarios where the system is idle and automatically scales the frequency of the vCPUs. Instead, T1 adopts a frequency scaling policy set by the users while executing its custom workflow.

Assessment – The HADDOCK team is generally *positive about the applicability* of this tactic. However, during our discussion, they were wary of the applicability of this tactic in those cases where the computational resources are shared by multiple workflows. Indeed, the concurrent execution of modules with different characteristics (*e.g.*, CPU-bound vs. bandwidth-bound) might require conflicting frequency adjustments. The team believes that integrating this tactic into the current HADDOCK code will require *a lot of effort*, especially when running HADDOCK workflows in environments where it is not possible to change the frequency scaling policy, such as in virtualized environments. This tactic seems original to the domain expert, while the product owner does not have a clear opinion about its originality since scaling the frequency is a common mechanism in the field, but not at the module level. Instead, for the developer, this tactic seems novel.

1.3 T3 Optimize Module Parameters

Short description – Tune the configuration of individual modules of a workflow based on their expected impact on resource usage and energy consumption.

Context – Software systems typically come with different configurations, which translate in a set of planned changes in the software functionality or architecture that can be tweaked at run-time [37]. Software systems that can be *reconfigured* provide a set of parameters to developers that can be applied before, during, or after the deployment of the software. Thus, reconfiguring the system helps better exploiting the resources of the available hardware platform with less effort than a full rebuild. For example, MySQL 5.6 provides 461 configuration parameters which allow the users to set parameters the supported data type, control buffer size, and resources limits [39] without rebuilding the system. It has been empirically proven that software configurations tend to be effective in optimizing resources utilization [7, 13, 25]; *e.g.*, execution time and energy consumption can be reduced using specific configurations [37]. Each module specified in an HADDOCK workflow comes with different parameters related to their functionality. For example, a user can specify a set of docking and clustering parameters that change the behavior of a module.

Description – Development teams shall compose workflows according to the impact of the parameters of the modules on performance and energy consumption. The tactic requires a controlled experiment to quantify how parameters influence performance and energy consumption. The results of the experiment can then be used to compose workflows optimized for resource utilization. This tactic is beneficial when the goal is to run workflows based on available resources. For example, it can be decided to delay the execution of heavy workloads and execute lighter ones when currently available resources are scarce. It is important to mention that some parameters are specific to molecular docking. Therefore, possible combinations and acceptable values of the parameter should be evaluated by a domain expert (*e.g.*, a bioinformatician).

Assessment – The HADDOCK team is *unsure about the applicability* of this tactic because implementing it would involve running a controlled experiment to evaluate the impact on resource utilization of the main parameters of all modules of HADDOCK. The team believes that this may not be feasible in their specific case. Below we report a quote summarizing this line of reasoning.

“If you are an expert on a specific topic, for example, lead and docking small molecules, you can do an experiment and see that you get good results using reduced sampling. Good enough, but it will be a result for a particular scenario and you will need to benchmark again to answer the next research question.” - Product Owner

Nevertheless, the HADDOCK team is confident that integrating such a tactic for specific cases would require relatively *little effort*, as the experiment can be carried out systematically for the most relevant parameters with relatively low effort (the team had already considered this policy in the past). However, they also mentioned that they did not implement it because there were too many

scenarios to evaluate in order to successfully integrate it in production. The latter point is hinting towards the necessity of a thorough trade-off analysis between the feasibility of running the preliminary experiment and the expected gains in terms of resource usage and energy improvements.

Finally, the HADDOCK team highlighted two additional points related to the tactic: (i) some parameters cannot be just tuned at will, but they have a specific semantics that is relevant for the workflow, and (ii) most HADDOCK users accept the default values provided by the team, so the importance of conducting a parameter-specific analysis might be overlooked.

1.4 T4 Use Resource-Aware Job Scheduling

Short description – Use job scheduling algorithms that consider resources like memory and energy, in addition to CPU usage only.

Context – Many job schedulers primarily focus on the CPU when assigning tasks within a cluster of servers. However, with the diversity of modern workloads, relying solely on CPU-based scheduling policies may lead to inefficiencies [21]. For example, memory-intensive workloads could significantly improve their performance by having access to more and faster memory (*e.g.*, a cache) to process large volumes of data efficiently [1, 23]. Thus, a scheduler should consider one or multiple resources, as CPU, memory, and energy, as well as workload characteristics when dispatching jobs to the servers. Multi-resources scheduling algorithms are well-known in the literature [11, 26, 33]. SLURM, which is one of the job schedulers used by HADDOCK and it is the one we used in our experiment, employs the **backfill** algorithm as the default job scheduler. The *backfill* algorithm considers a limited set of resources to evaluate how to dispatch the jobs, such as CPU and memory requirements¹ but it can be easily extended to incorporate more resources. For example, the algorithm proposed by D’Amico and Gonzalez [9] extends the **backfill** algorithm by (i) using a model to estimate the energy requirements of the jobs, and (ii) setting the priority of the jobs in the queue according to energy estimates. Resource-aware scheduling algorithms can be used to improve the performance and energy efficiency of HPC software like HADDOCK.

Description – Development teams shall use resource-aware scheduling techniques considering single or multiple resources, both when submitting and when migrating jobs. Depending on where optimization is to be applied, the workload manager (*e.g.*, SLURM) shall incorporate mechanisms to know the expected resource utilization of the jobs being scheduled. For example, if the scheduling algorithm aims to optimize resources at job submission, then it needs a technique for estimating the resource utilization of queued jobs. Also, the job scheduler should be integrated with a monitoring system so to be able to check available resources in the cluster and those used by individual jobs.

Assessment – The domain expert and the software developer are *strongly convinced of the applicability* of this tactic in HADDOCK, while the product owner

¹https://slurm.schedmd.com/sched_config.html

had a neutral stance. All three practitioners state that HADDOCK users can specify the resources to allocate to their jobs through (i) job files (*i.e.*, slurm scripts) and (ii) workflow configuration files. Thus, this tactic is already partially implemented in HADDOCK. However, users often do not know the amount of resources required by their custom workflows, leading to a risk of over- or under-estimating their needed resources. The product owner described a situation in which a workflow required more network bandwidth to write files. By quoting the HADDOCK product owner:

“[A user] was running 12 jobs with eight cores on a node. This was where we had an NFS issue that was creating a bottleneck in the network. The execution was stuck in a waiting phase due to the creation of a lot of files“ - Product Owner

The product owner has indicated the necessity for a scheduling algorithm that efficiently fills all available resources. Unused resources during idle times result in wasted energy consumption and higher costs. Currently, the HADDOCK team uses a round-robin policy to allocate jobs to nodes and they are seeking for a more efficient scheduling approach that can save energy and performance. By quoting the product owner:

“You have this baseline energy consumption where you pay energy for nothing. SLURM allows you to shut down and restart nodes when needed. So, this can be another way to save energy. [...] You fill the nodes and if they are not used for 5 minutes, we shut them down and when there is demand again we restart when. It should not introduce much overhead since [restarting the nodes] might be very fast. Maybe one or two minutes.“ - Product Owner

The HADDOCK team assessed the expected effort and the originality of this tactic as low since they already have in place a resource-aware technique to job scheduling. However, since at the current state they use a simple algorithm (*i.e.*, round-robin), they need a more efficient policy that introduces greater improvements in the performance and energy efficiency of HADDOCK.

1.5 T5 Adjust CPU Load Balancing

Short description – Use CPU process schedulers like EAS and CAS that consider resources usage and energy, in addition to CPU time only.

Context – Optimized resource utilization can be achieved by effectively load balancing the workload assigned to the vCPUs of a server. Since version 6.6, Linux implements the Earliest Eligible Virtual Deadline First Scheduling (EEVDF) algorithm, while prior to that version Linux implemented a Completely Fair Scheduler (CFS). Both schedulers focus on providing the same share of CPU time to all tasks ². Energy usage is not a primary concern for EEVDF, which instead focuses on latency.

²<https://docs.kernel.org/scheduler/index.html>

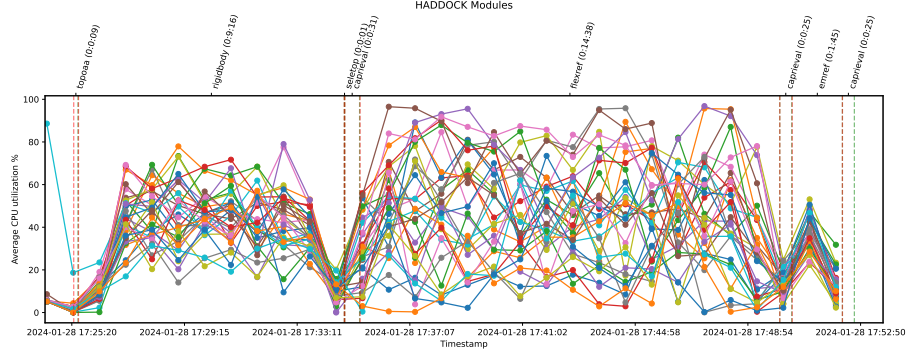


Figure 2: CPUs Utilization (%) Trace for DPP executed with 16 active vCPUs.

The Linux kernel offers alternatives to the EEVDF, such as the Energy-Aware Scheduling algorithm (EAS), which prioritizes energy efficiency by utilizing an energy model to forecast the most energy-efficient CPU allocation. Another option is the Capacity-Aware Scheduling algorithm (CAS), which takes into account task requirements, such as CPU utilization. There are many studies that provide enhancements to Linux schedulers, *e.g.*, [8, 19, 20, 22]. Among them, Lawall et al. [20] present NEST, an extension of CFS that aims to reduce idle times of the cores. The results show performance and energy improvements for applications using fewer tasks than cores.

Description – Development teams shall choose scheduling algorithms consciously, in such a way that tasks are assigned to vCPUs according to the characteristics of the tasks and their energy requirements. Example of promising scheduling algorithms are EAS and CAS. An ideal scheduling algorithm should estimate the CPU utilization of a task and, if it is high, assign the task to the most available vCPU, or find a way to occupy the idle vCPUs while waiting for memory response. More complex solutions can include grouping the vCPUs and allocating them to different types of tasks. For example, more vCPUs can be dedicated to resource-greedy tasks, while the remaining ones can be shared among lightweight tasks (*i.e.*, code-scheduling, gang scheduling)³. In this study, we conduct our experiment using the CFS scheduler. As shown in Figure 2, the vCPUs utilization trend is unpredictable as the scheduler assigns CPU time fairly to all the vCPUs. Dynamic vCPUs Load Balancing requires a precise monitoring system that can track CPU usage for individual tasks, even for short-lived ones. However, detailed monitoring could lead to increased overhead as a result of frequent requests to the kernel for vCPUs statistics.

Assessment – The HADDOCK team agrees on the *minimal applicability* and the *extensive originality* of this tactic. Both the software developer and the product owner mention that this tactic can be problematic in the context of HADDOCK as sub-tasks have similar computational demands and can be useful

³<https://docs.kernel.org/admin-guide/hw-vuln/core-scheduling.html>

for more diverse workflows. Indeed, the distinction between heavy and light tasks is not prominent in HADDOCK. The domain expert remarks that such an approach could lead to different optimizations varying the underlying hardware platform. Indeed, different hardware platforms present different numbers of vCPUs with diverse frequency and threading. This comment is supported by the product owner, who suggests that the benefits of dynamic load balancing could be more evident in heterogeneous clusters with nodes offering different resources and when using threading, *e.g.*, running Message Passing Interface (MPI) threads.

According to the software developer, this tactic could be the easiest to implement (*low effort*) since they already have a way to manage tasks in HADDOCK. They already have a system in place to execute tasks in HADDOCK, which queues tasks and allocates one task per vCPU as soon as a vCPU becomes available. Thus, HADDOCK already has a component that implements task allocation that can be independently changed to implement a different allocation strategy. However, monitoring the vCPUs with a fixed polling rate can introduce multiple challenges, such as tracking individual task executions on different vCPUs and monitoring overhead. By quoting the HADDOCK software developer:

“We need a way to check what each vCPU is performing because we have already a queue (of tasks) of what should be delivered to each vCPU. We just need to decide how to assign them and how we are going to order them. So, where each task goes, we also have a way to estimate its complexity“ - Software Developer

1.6 T6 Adopt Malleable Jobs

Short description – Adopt malleable jobs to achieve lower utilization of resources (including energy), lower response times, and lower waiting times.

Context – Most batch systems, such as SLURM, used by HADDOCK and in our experiment, only support rigid jobs. SLURM allocates a fixed amount of resources (*e.g.*, the number of vCPUs) for the whole execution of the job.

Description – Development teams shall transition from fixed-size jobs to malleable jobs. *Malleable jobs* [12] can adapt their resource utilization at runtime, *e.g.*, they can dynamically increase or reduce the number of used vCPUs or nodes based on available resources. Malleable jobs have been successfully adopted in the past, and their positive impact on performance and energy consumption has been empirically proven [24, 28, 36]. Chadha et al. [5] extend SLURM to support adaptive performance and power-aware management of jobs. The authors introduce a software component called Adaptive Batch Scheduler (ABS) that monitors job execution and available system resources. Hence, the ABS communicates with SLURM to indicate which jobs should undergo expand/shrink operations. Chadha *et al.* compare the benefits of performance and energy efficiency of their approach to common scheduling algorithms, such as static-backfill and FPSMA policies. The results show that by using mal-

leable jobs, it is possible to achieve lower response and waiting time, as well as a lower utilization of resources (*i.e.*, fewer idle nodes). Their approach enables the system involved in the experiment to maintain power consumption within a specific range, thereby avoid power outage.

Currently HADDOCK divides tasks into fixed-sized batches, based on the number of available vCPUs. However, dynamically adjusting the resources allocated to a batch according to the current system load and resource availability could significantly improve workflow execution time and energy efficiency. For instance, as shown in Figure 2, the number of allocated resources can be increased when running resource-intensive modules like `rigidbody` and `flexref` and reduced for `topoaa`. As for tactic T3, this tactic assumes knowledge regarding the resource utilization of the modules and their dependencies. It is important to clarify that this tactic differs from the T4 tactic as follows: in the case of T3 we propose to dynamically change the resources adaptively allocated to jobs, whereas in T4 we propose to change the way the jobs are ordered following a pre-defined policy (*e.g.*, to give more priority to jobs with less execution time) and change the resources allocated to jobs.

Assessment – The HADDOCK team firmly believes that enabling malleable jobs *can be applicable* in the context of HADDOCK and that it can be fruitful to optimize the performance and energy efficiency. However, using parallelization (*e.g.*, threading) to optimize resource utilization would be their preferred approach, before considering dynamic execution adaptation. Since the modules of HADDOCK are primarily CPU-bound, they aim to maximize server resource utilization rather than introducing the complexity that dynamic execution adaptation would bring in a distributed setting. The same observation has been made for the tactic T1. In addition, the team remarks that the integration of such a tactic *may not require much effort* due to the architecture of HADDOCK. In fact, HADDOCK already includes a mechanism to monitor the resources used by the batches of jobs. However, the HADDOCK team seems to be skeptical about the potential overhead introduced by the usage of malleable jobs, such as the overhead introduced by polling the batches to be scheduled. As summarized by the HADDOCK product owner:

“You need to take into account the CPU utilization derived by the monitoring phase. In some cases, you need to track 10000 models and you do not want to check too often to avoid overhead.” - Product Owner

The HADDOCK team *does not deem this tactic as original*. They had already considered it but did not implement it due to low applicability in their context and possible overhead derived from the monitoring phase.

1.7 T7 Enable CPU Oversubscription

Short description – Enable (energy-aware) CPU oversubscription, possibly paired with a monitoring infrastructure to control and adjust its overhead.

Context – Multithreading has been proven effective in optimizing performance and energy consumption [3, 10, 27, 31, 34]. Its benefits are notable for applications generating heavy workloads, such as scientific applications [4]. Heavy workloads can be divided into independent subtasks embodied by software threads, ultimately distributed across the cores using the process scheduler of the operating system. Linux uses several schedulers, including the Completely Fair Scheduler (CFS), to distribute tasks across the cores. As mentioned for T5, Linux schedulers can be designed to improve the utilization of specific resources, such as energy. However, they can be inefficient in highly parallel and resource-competitive environments [22]. CPU oversubscription can be beneficial in such cases. CPU oversubscription refers to the practice of having more tasks than the available cores [17]. Iancu et al. [16] show that oversubscription can improve the overall system throughput by 27% when resources are shared by multiple applications. Given that parallelized applications can generate more threads than the underlying hardware can effectively manage, Huang et al. [15] explored CPU oversubscription as a strategy to optimize core utilization. In particular, they investigated the causes of overhead introduced by CPU oversubscription, such as context switching, loss of memory locality, and resource contention, and proposed approaches to address the mentioned slowdowns. The results indicate that employing oversubscription can enhance CPU utilization, while maintaining overhead levels comparable to those found in undersubscribed scenarios.

Description – Development teams shall enable CPU oversubscription, possibly paired with a monitoring infrastructure to control and adjust its overhead. Scientific software is usually designed to allocate one task per core and execute on different hardware platforms [15, 22]. In our experiment, HADDOCK statically assigns a single task to a single vCPU. Thus, CPU oversubscription is a promising solution in the context of HADDOCK if: (i) a user needs to run multiple workflows at the same time and (ii) the resources used by HADDOCK are also being used by other applications. Figure 2 represents the CPU utilization distribution of a workflow and shows some dips when executing lighter modules, such as `seletop` and `caprieval`. The (unused) resources present in those dips can be used by the tasks of other applications or workflows.

Assessment – The HADDOCK team is *overall positive* about the applicability of this tactic. The product owner remarks that in the version used by our experiment, HADDOCK limits the creation (and consequently the allocation) of the number of tasks to the total number of cores. In addition, from their experience, the applicability of this tactic will be influenced by the hardware in which it is applied, as different systems could implement different threading and scheduling approaches that may affect the overhead of CPU oversubscription. Even if CPU oversubscription can have benefits in some cases, the team HADDOCK highlights that, to their knowledge, there is no evidence on the effect that the application of this tactic could have on the energy consumption. CPU oversubscription would need a fine-grained monitoring systems that is capable of adjusting the allocation of the threads to keep threading overhead at a minimum, as done in [17].

The HADDOCK team agrees on the *extensive effort* needed to implement this tactic. They propose that CPU oversubscription could be implemented in the workload manager (*e.g.*, SLURM), that could allow them to assign more slots than cores for a workflow. The HADDOCK team *do not comment on the originality* of this tactic, as they already have a code section that manages the allocation of tasks to cores. In the version of HADDOCK evaluated in our experiment, something similar is done at task-level using a functionality of HADDOCK called *concatenation*, which allows the user to pack more models in a single thread.

1.8 T8 Increase Task-level Parallelism

Short description – Structure the workflow so that tasks represent the smallest unit of parallelizable functionality, with each task executing in its own thread.

Context – Heavy workloads can be decomposed into smaller independent sub-tasks and executed in parallel using threads. This practice is commonly known as *task-level parallelization* [30, 31] and consists of refactoring an application to the smallest unit of parallelizable functionality. Task-level parallelization differs from thread-level parallelization, in which multiple threads compete for the same resources and can handle multiple tasks [35, 40]. Specifically, refactoring an application to obtain the optimal count of tasks can be more efficient than just increasing the number of threads since adding threads introduces overhead due to thread management and contention of shared resources [29]. Differently, introducing optimal parallelism at the task level can help achieve the optimal thread count, *i.e.*, the number of threads that do not introduce performance degradation, and thus improve the resource utilization of the application, as the number of threads will be optimized according to the number of tasks of the application. Chasapis et al. [6] applied 10 task-level parallelization strategies to 10 applications achieving an average speedup improvement of 13%.

Description – HADDOCK3 divides the tasks of a module across parallel processes. In particular, multiple tasks can be assigned to a single process. In T8, we propose to increase the level of task parallelism and use an approach to find the optimal number of tasks that can be executed in parallel without performance degradation. In our experiment, we noticed that there is room to further parallelize the analysis modules that perform heavy computations, *e.g.*, matrix multiplications. However, the application of this tactic comes with the challenge of deciding what are the parallelizable tasks. Indeed, tasks can depend on each other. CUDA provides ways to manually define tasks (*e.g.*, using threads) and distribute them on the GPU ⁴. This process can be non-trivial if it is not supported by functional decomposition approaches, such as the adoption of dependency visualization methods (*e.g.*, flowcharts).

Assessment – The HADDOCK team *agrees about the applicability* of this tactic. They are evaluating to apply it, but it requires *a lot of effort* because of the HADDOCK architecture. The code evaluated in this experiment contains

⁴<https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model/>

a base class to create a module that does not allow parallelization. This means that parallelization is implemented for a subset of the modules, only where parallelization is implemented in the module-specific source code. The software developer mentions that parallelization should be kept in one place (*i.e.*, in the base class that all modules extend) to avoid redundant code. However, the developer remarks that they are not planning to do that due to the extensive refactoring needed. The product owner mentions that this could be done in the next version of HADDOCK.

“It is complicated because it comes [the effort] from the way the code is structured. [...] [`caprieval` is parallelized] using a different class and the base classes are not parallelized, that is what we need to change. [...] We can just make it in one place [parallelization] but this is not going to be too simple. It will change the code a lot, [...] I think it is too late for it.” - Software Developer

The team confirmed that this tactic has *poor originality* since they have already thought about it. In fact, some module implements a mechanism to increase task-level parallelism using the *concatenation* parameter in the workflow configuration file, which allows HADDOCK to simulate multiple models at the same time.

References

- [1] Kazi Asifuzzaman, Narasinga Rao Miniskar, Aaron R Young, Frank Liu, and Jeffrey S Vetter. A survey on processing-in-memory techniques: Advances and challenges. *Memories-Materials, Devices, Circuits and Systems*, 4:100022, 2023.
- [2] Wenlei Bao, Changwan Hong, Sudheer Chunduri, Sriram Krishnamoorthy, Louis-Noël Pouchet, Fabrice Rastello, and P Sadayappan. Static and dynamic frequency scaling on multicore cpus. *ACM Transactions on Architecture and Code Optimization (TACO)*, 13(4):1–26, 2016.
- [3] Geoffrey Blake, Ronald G Dreslinski, Trevor Mudge, and Krisztián Flautner. Evolution of thread-level parallelism in desktop applications. In *Proceedings of the 37th annual international symposium on Computer architecture*, pages 302–313, 2010.
- [4] John M Campbell, R Keith Ellis, and Walter T Giele. A multi-threaded version of mcfm. *The European Physical Journal C*, 75:1–7, 2015.
- [5] Mohak Chadha, Jophin John, and Michael Gerndt. Extending slurm for dynamic resource-aware adaptive batch scheduling. In *2020 IEEE 27th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, pages 223–232. IEEE, 2020.

- [6] Dimitrios Chasapis, Marc Casas, Miquel Moretó, Raul Vidal, Eduard Ayguadé, Jesús Labarta, and Mateo Valero. Parsecss: Evaluating the impact of task parallelism in the parsec benchmark suite. *ACM Transactions on Architecture and Code Optimization (TACO)*, 12(4):1–22, 2015.
- [7] Tao Chen and Miqing Li. Multi-objectivizing software configuration tuning. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, page 453–465, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450385626. doi: 10.1145/3468264.3468555. URL <https://doi.org/10.1145/3468264.3468555>.
- [8] Hojin Cho and Myungsun Kim. gcfs: completely fair scheduling on multiple gpus for improved multi-dnn execution in terms of performance isolation. *The Journal of Supercomputing*, 79(6):5851–5877, 2023.
- [9] Marco D’Amico and Julita Corbalan Gonzalez. Energy hardware and workload aware job scheduling towards interconnected hpc environments. *IEEE Transactions on Parallel and Distributed Systems*, 2021.
- [10] Danny Dig. A refactoring approach to parallelism. *IEEE software*, 28(1):17–22, 2010.
- [11] Yuping Fan, Zhiling Lan, Paul Rich, William E Allcock, Michael E Papka, Brian Austin, and David Paul. Scheduling beyond cpus for hpc. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, pages 97–108, 2019.
- [12] Abhishek Gupta, Bilge Acun, Osman Sarood, and Laxmikant V Kalé. Towards realizing the potential of malleable jobs. In *2014 21st International Conference on High Performance Computing (HiPC)*, pages 1–10. IEEE, 2014.
- [13] Xue Han and Tingting Yu. An empirical study on performance bugs for highly configurable software systems. In *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ESEM ’16, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450344272. doi: 10.1145/2961111.2962602. URL <https://doi.org/10.1145/2961111.2962602>.
- [14] Darong Huang, Luis Costero, and David Atienza. Is the powersave governor really saving power? In *2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pages 273–283. IEEE, 2024.
- [15] Hang Huang, Jia Rao, Song Wu, Hai Jin, Hong Jiang, Hao Che, and Xiaofeng Wu. Towards exploiting cpu elasticity via efficient thread oversubscription. In *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing*, pages 215–226, 2021.

- [16] Costin Iancu, Steven Hofmeyr, Filip Blagojević, and Yili Zheng. Oversubscription on multicore processors. In *2010 IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*, pages 1–11. IEEE, 2010.
- [17] Pierre Jacquet, Thomas Ledoux, and Romain Rouvoy. Sweetspotvm: Oversubscribing cpu without sacrificing vm performance. In *24th IEEE/ACM international Symposium on Cluster, Cloud and Internet Computing (CC-Grid)*. IEEE, 2024.
- [18] Michał P Karpowicz. Energy-efficient cpu frequency control for the linux system. *Concurrency and Computation: Practice and Experience*, 28(2): 420–437, 2016.
- [19] Jungho Kim, Philkyue Shin, Myungsun Kim, and Seongsoo Hong. Memory-aware fair-share scheduling for improved performance isolation in the linux kernel. *IEEE Access*, 8:98874–98886, 2020.
- [20] Julia Lawall, Himadri Chhaya-Shailesh, Jean-Pierre Lozi, Baptiste Lepers, Willy Zwaenepoel, and Gilles Muller. Os scheduling with nest: Keeping tasks close together on warm cores. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 368–383, 2022.
- [21] Boyang Li, Yuping Fan, Matthew Dearing, Zhiling Lan, Paul Rich, William Allcock, and Michael Papka. Mrsch: Multi-resource scheduling for hpc. In *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 47–57. IEEE, 2022.
- [22] Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. The linux scheduler: a decade of wasted cores. In *Proceedings of the Eleventh European Conference on Computer Systems*, pages 1–16, 2016.
- [23] Hosein Mohammadi Makrani, Hossein Sayadi, Sai Manoj Pudukotai Dinakarra, Setareh Rafatirad, and Houman Homayoun. A comprehensive memory analysis of data intensive workloads on server class architecture. In *Proceedings of the International Symposium on Memory Systems*, pages 19–30, 2018.
- [24] Ao Mo-Hellenbrand, Isaías Comprés, Oliver Meister, Hans-Joachim Bungartz, Michael Gerndt, and Michael Bader. A large-scale malleable tsunami simulation realized on an elastic mpi infrastructure. In *Proceedings of the Computing Frontiers Conference*, pages 271–274, 2017.
- [25] Vivek Nair, Zhe Yu, Tim Menzies, Norbert Siegmund, and Sven Apel. Finding faster configurations using flash. *IEEE Transactions on Software Engineering*, 46(7):794–811, 2018.
- [26] Lucas Perotin, Sandhya Kandaswamy, Hongyang Sun, and Padma Raghavan. Multi-resource scheduling of moldable workflows. *Journal of Parallel and Distributed Computing*, 184:104792, 2024.

- [27] Vinicius Petrucci, Orlando Loques, Daniel Mossé, Rami Melhem, Neven Abou Gazala, and Sameh Gobriel. Energy-efficient thread assignment optimization for heterogeneous multicore systems. *ACM Transactions on Embedded Computing Systems (TECS)*, 14(1):1–26, 2015.
- [28] Suraj Prabhakaran, Marcel Neumann, Sebastian Rinke, Felix Wolf, Abhishek Gupta, and Laxmikant V Kale. A batch system with efficient adaptive scheduling for malleable and evolving applications. In *2015 IEEE international parallel and distributed processing symposium*, pages 429–438. IEEE, 2015.
- [29] Kishore Kumar Pusukuri, Rajiv Gupta, and Laxmi N Bhuyan. Thread re-inforcer: Dynamically determining number of threads via os level monitoring. In *2011 IEEE International Symposium on Workload Characterization (IISWC)*, pages 116–125. IEEE, 2011.
- [30] Robert Robey and Yuliana Zamora. *Parallel and high performance computing*. Simon and Schuster, 2021.
- [31] Daniele Paolo Scarpazza, Praveen Raghavan, David Novo, Francky Catthoor, and Diederik Verkest. Software simultaneous multi-threading, a technique to exploit task-level parallelism to improve instruction-and data-level parallelism. In *Integrated Circuit and System Design. Power and Timing Modeling, Optimization and Simulation: 16th International Workshop, PATMOS 2006, Montpellier, France, September 13-15, 2006. Proceedings 16*, pages 12–23. Springer, 2006.
- [32] Claudio Scordino, Luca Abeni, and Juri Lelli. Real-time and energy efficiency in linux: theory and practice. *ACM SIGAPP Applied Computing Review*, 18(4):18–30, 2019.
- [33] Mehdi Sheikhalishahi, Richard M Wallace, Lucio Grandinetti, José Luis Vazquez-Poletti, and Francesca Guerriero. A multi-dimensional job scheduling. *Future Generation Computer Systems*, 54:123–131, 2016.
- [34] Won So and Alexander G Dean. Software thread integration for instruction-level parallelism. *ACM Transactions on Embedded Computing Systems (TECS)*, 13(1):1–23, 2013.
- [35] Nikki Sonenberg, Grzegorz Kielanski, and Benny Van Houdt. Performance analysis of work stealing in large-scale multithreaded computing. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 6(2):1–28, 2021.
- [36] Zhe Wang, Matthieu Dorian, Pradeep Subedi, Philip E Davis, and Manish Parashar. Adaptive elasticity policies for staging-based in situ visualization. *Future Generation Computer Systems*, 142:75–89, 2023.

- [37] Max Weber, Christian Kaltenecker, Florian Sattler, Sven Apel, and Norbert Siegmund. Twins or false friends? a study on energy consumption and performance of configurable software. In *Proceedings of the 45th International Conference on Software Engineering, ICSE*, volume 23, page 12, 2023.
- [38] Rafael J. Wysocki. Cpu performance scaling, 2017. URL <https://docs.kernel.org/admin-guide/pm/cpufreq.html>. Accessed: 2024-07-27.
- [39] Tianyin Xu, Long Jin, Xuepeng Fan, Yuanyuan Zhou, Shankar Pasupathy, and Rukma Talwadker. Hey, you have given me too many knobs!: Understanding and dealing with over-designed configuration in system software. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 307–319, 2015.
- [40] Qingling Zhao, Yecheng Zhao, Minhui Zou, Zhigang Gao, and Haibo Zeng. Improved analysis and optimal priority assignment for communicating threads on uni-processor. *Journal of Systems Architecture*, 124:102377, 2022.